

Pós-Graduação em Engenharia de Dados

Pipelines ETL com AWS Glue Studio

Nadia Guimaraes

Porto Alegre, novembro de 2026

Sumário

Introdução 3
Enunciado 3
Desenvolvimento 3
  Segurança 4
    Configurações de acesso IAM 4
    Configuração de usuários, funções e políticas 4
  Dados 5
    Organização do armazenamento S3 5
    Organização na arquitetura "Medallion" 5
    Fontes de dados variadas (JSON, CSV, Parquet) 6
    Configurações com foco na redução de custos 6
    Sugestão de como tratar "arquivos pequenos" 7
  Ingestão e Transformação 7
    Configuração no AWS Glue Data Catalog com Crawlers 7
    Fonte JDBC: cadastros em um banco RDS PostgreSQL 8
    Configuração no AWS Glue de algumas transformações 10
    Configuração no AWS Glue convertendo para Parquet 11
  Eficiência e Performance 11
    Configuração no AWS Glue do Job Bookmarks 11
    Captura de mudanças (CDC) da fonte JDBC 12
    Configuração do agendamento dos Jobs 13
  Governança, segurança e qualidade dos dados 14
    Configuração da proteção de informações confidenciais 14
    No AWS Glue e AWS Glue DataBrew 14
Conclusão 16
Referências 16

Introdução

Este trabalho descreve a construção de um pipeline de dados na AWS, da ingestão dos arquivos brutos até um modelo analítico consultado pelo Amazon Athena. Foi o meu primeiro contato prático com o AWS Glue, então procurei registrar não só o resultado final, mas também os erros que encontrei e como os resolvi.

O cenário escolhido é um centro de manutenção de aeronaves (MRO), que registra cada compra de ferramenta como uma fatura ligada à aeronave atendida. A partir dos três arquivos fornecidos pela disciplina (faturas em JSON, produtos em JSON e aviões em CSV), gerei um conjunto maior, com **1.208.246 faturas em 5.739 arquivos**, para ter volume suficiente para testar o Glue. Os dados foram divididos em dois lotes (janeiro a junho e julho a agosto de 2025) para demonstrar o processamento incremental.

As etapas foram: criação da base com CloudFormation (bucket S3, permissões IAM, bancos do Data Catalog, crawlers e workgroup do Athena); ingestão na camada Bronze e descoberta do esquema com crawler; dois jobs visuais no Glue Studio para gerar a camada Silver; um job Spark para o modelo estrela da camada Gold; orquestração com Glue Workflow; e validação com consultas no Athena. Depois, para completar os requisitos, acrescentei o perfil de dados e uma receita de limpeza no **AWS Glue DataBrew**, um banco **RDS PostgreSQL** como fonte **JDBC** dos cadastros e uma captura de mudanças (**CDC**) desse banco para a Silver. Tudo foi executado na região **us-east-2 (Ohio)**.

Enunciado

O projeto pede o desenho e a implementação de um pipeline de dados distribuído que integre ingestão e transformação com o AWS Glue Studio e governança centralizada no Glue Data Catalog. Devem ser entregues:

- **Trabalhos do AWS Glue (jobs):** configuração e estruturação dos bancos de dados e tabelas no Glue Data Catalog, com o mapeamento de metadados e a definição das entidades das camadas de ingestão e processamento.
- **Configuração de governança e metadados:** documentação ou capturas de tela da organização das permissões, dos catálogos e da administração de metadados técnicos.
- **Relatório técnico (PDF):** arquitetura utilizada, justificativa da organização do armazenamento em camadas no S3, análise de custos estimada e métricas de monitoramento observadas.

Desenvolvimento

A visão geral da solução está na figura abaixo. Os dados passam por três camadas no S3 (Bronze, Silver e Gold), o Glue processa cada passagem e o Data Catalog registra o esquema de cada camada.

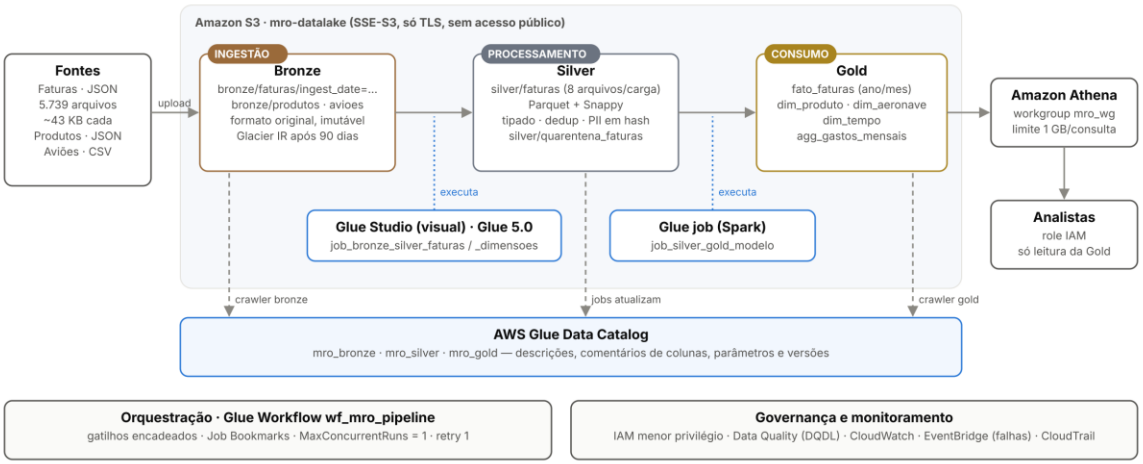


Figura 1 – Arquitetura da solução: Data Lake em camadas no S3, processamento no Glue e metadados no Data Catalog

Segurança

Configurações de acesso IAM

Os jobs e crawlers usam a role **mro-glue-service-role**, criada com o princípio do menor privilégio: ela lê a camada Bronze e os scripts e só grava nas camadas Silver e Gold. Além dela, a role recebeu a política gerenciada `AWSGlueServiceRole`, com o básico para o serviço funcionar. Os dados gravados pelos jobs são criptografados com SSE-S3 pela configuração de segurança `mro-sc-sse-s3`, e o bucket nega acesso público e requisições sem TLS.

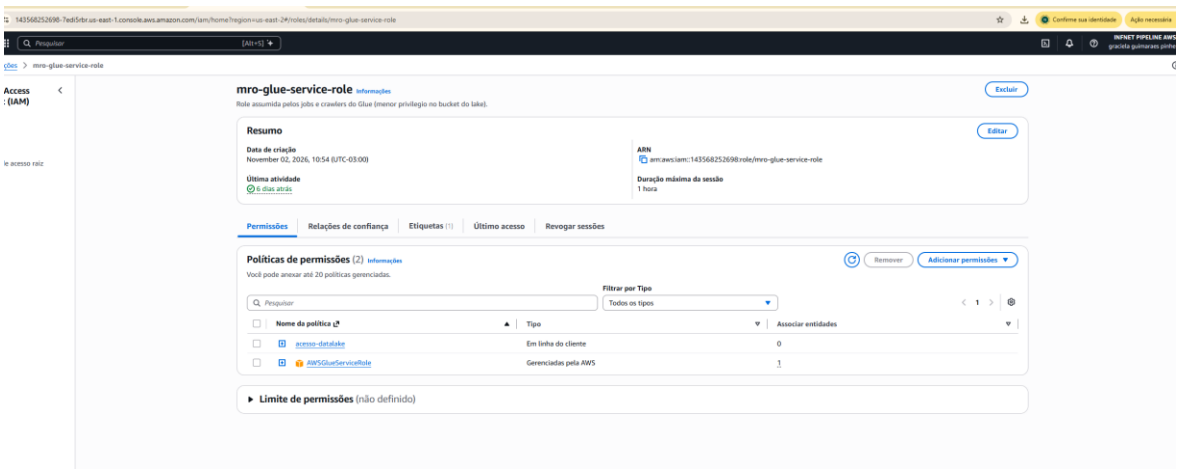


Figura 2 – Role mro-glue-service-role com a política em linha acesso-datalake e a política gerenciada do Glue (imagem 50)

Configuração de usuários, funções e políticas

Para os analistas, o plano era um grupo IAM. A criação falhou porque a organização da conta tem uma política de controle de serviço (SCP) que bloqueia `iam:CreateGroup`. Troquei o grupo pela role **mro-analista-role**, com a mesma política: o analista consulta pelo Athena e lê somente a camada Gold, sem acesso à Bronze nem à Silver, onde estão os dados brutos e pessoais.

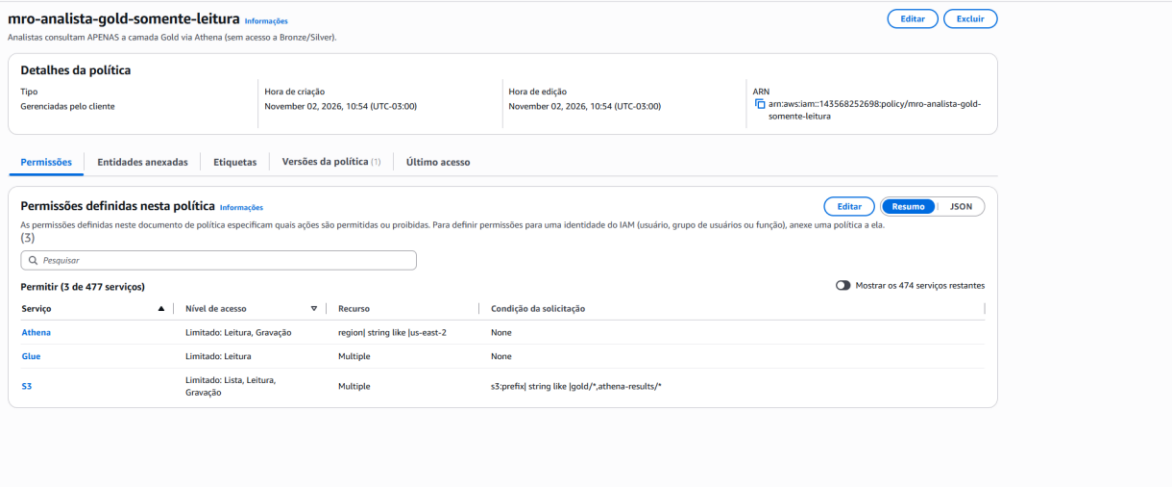


Figura 3 – Política mro-analista-gold-somente-leitura: Athena na região, Glue só leitura e S3 restrito a gold/ e athena-results/ (imagem 52)

Dados

Organização do armazenamento S3

Usei um único bucket, `mro-datalake-143568252698-us-east-2`, dividido em prefixos por camada (`bronze/`, `silver/`, `gold/`) e por apoio (`scripts/`, `temp/`, `athena-results/`). Na Bronze, os arquivos de faturas ficam particionados pela data de chegada (`ingest_date=AAAA-MM-DD`), o que permite ao crawler e ao Job Bookmark processar só as pastas novas.

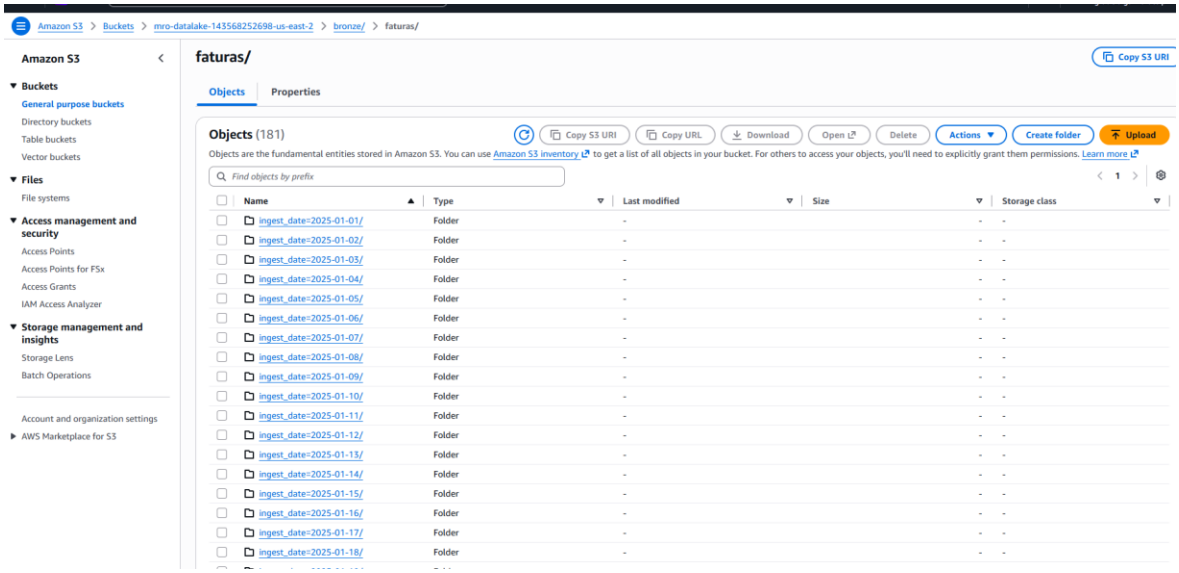


Figura 4 – Camada Bronze: bronze/faturas/ particionada por ingest_date (imagem 10)

Organização na arquitetura "Medallion"

Cada camada tem um papel e um banco próprio no Data Catalog. A **Bronze** guarda os dados como chegaram, sem alteração, para auditoria e reprocessamento. A **Silver** tem os dados limpos, tipados, sem duplicados e com o e-mail protegido. A **Gold** tem o modelo estrela (um fato e três dimensões) e um agregado mensal, prontos para análise.

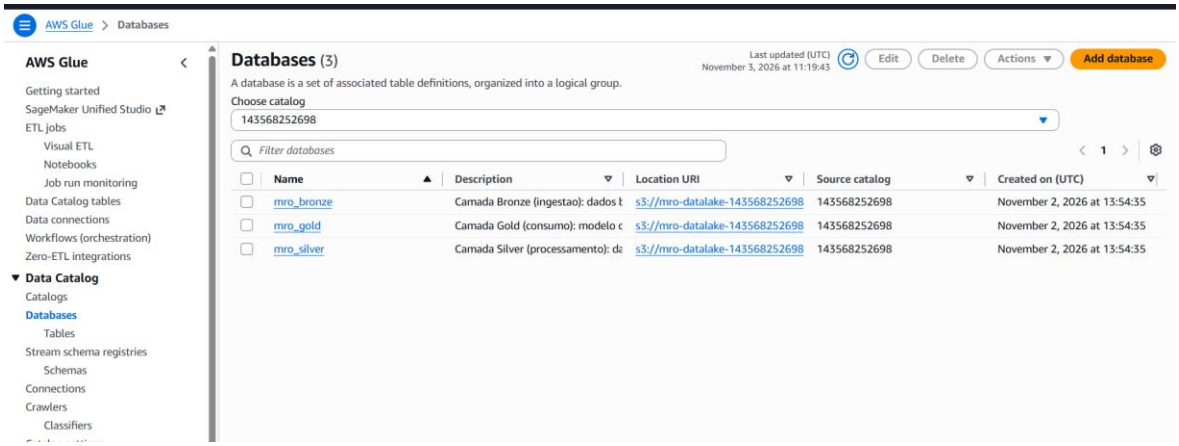


Figura 5 – Bancos mro_bronze, mro_silver e mro_gold no Glue Data Catalog, com descrição e local no S3 (imagem 02)

Fontes de dados variadas (JSON, CSV, Parquet)

A Bronze recebe faturas e produtos em JSON Lines e aviões em CSV com cabeçalho. A partir da Silver, tudo é gravado em Parquet com compressão Snappy.

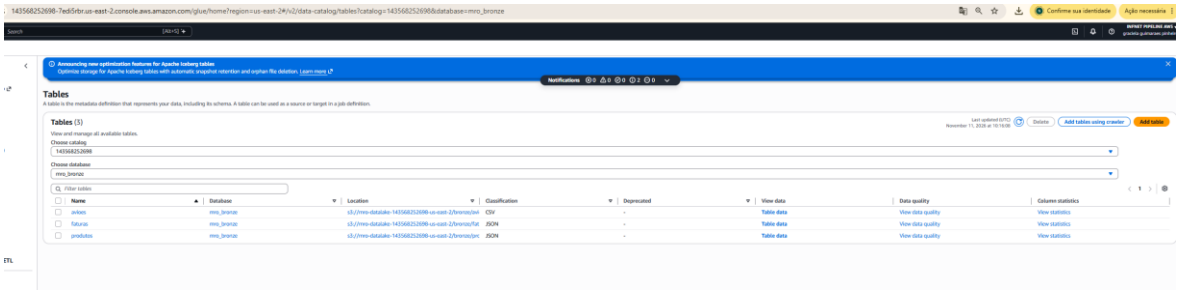


Figura 6 – Tabelas da Bronze: avioes em CSV, faturas e produtos em JSON (imagem 53)

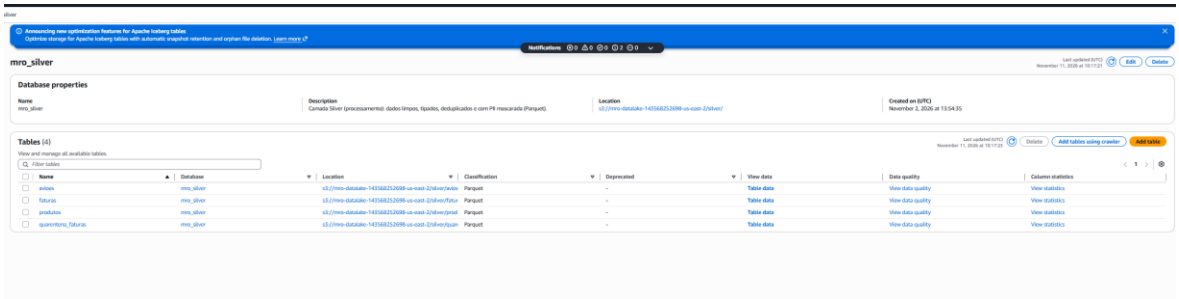


Figura 7 – Tabelas da Silver, todas em Parquet (imagem 54)

Configurações com foco na redução de custos

- **Parquet + Snappy:** a Silver de faturas ocupa 21,5 MB, contra 164 MB de JSON na Bronze (cerca de 85% menos).
- **Ciclo de vida no S3:** a Bronze vai para Glacier Instant Retrieval após 90 dias; temporários e resultados do Athena expiram sozinhos.
- **Partição por ano e mês na Gold** e limite de 1 GB lido por consulta no workgroup `mro_wg` do Athena.
- **Jobs com 2 workers G.1X** (o console sugeria 40) e no máximo 1 execução por vez.

No Athena, a mesma pergunta (gasto de março) leu 527,83 KB com filtro de partição e 616,71 KB sem ele. A diferença é pequena porque o Parquet já lê só as colunas usadas; em volumes maiores, a partição faz mais diferença.

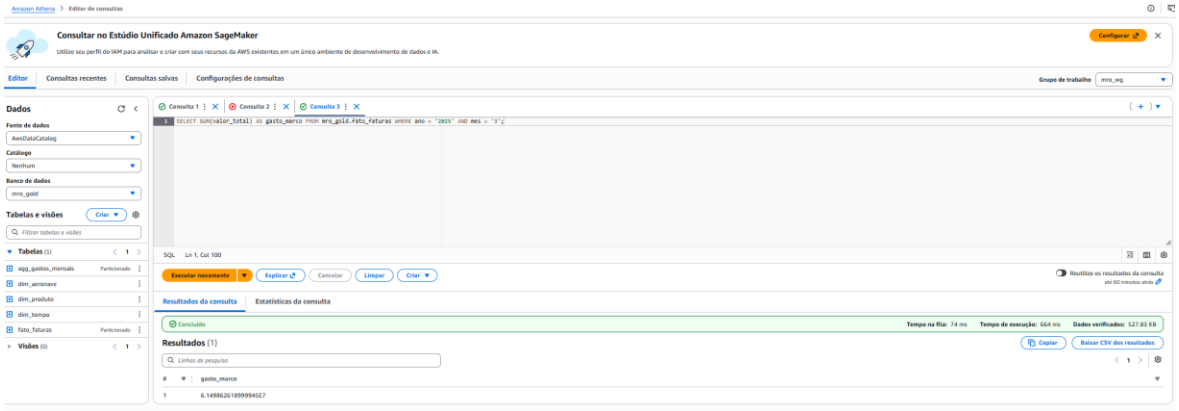


Figura 8 – Consulta com filtro de partição: 527,83 KB verificados (imagem 45)

Sugestão de como tratar "arquivos pequenos"

A Bronze recebe um arquivo por hora, com cerca de 43 KB cada (24 por dia). Tratei o problema nas duas pontas: na leitura, as propriedades `groupFiles=inPartition` e `groupSize=128 MB` na tabela fazem o Glue agrupar os arquivos de cada partição; na escrita, o nó Autobalance Processing grava poucos arquivos grandes. O resultado foi 8 arquivos Parquet por carga, contra 4.272 arquivos JSON no lote 1. Em produção, eu também compactaria a Bronze em arquivos diários.

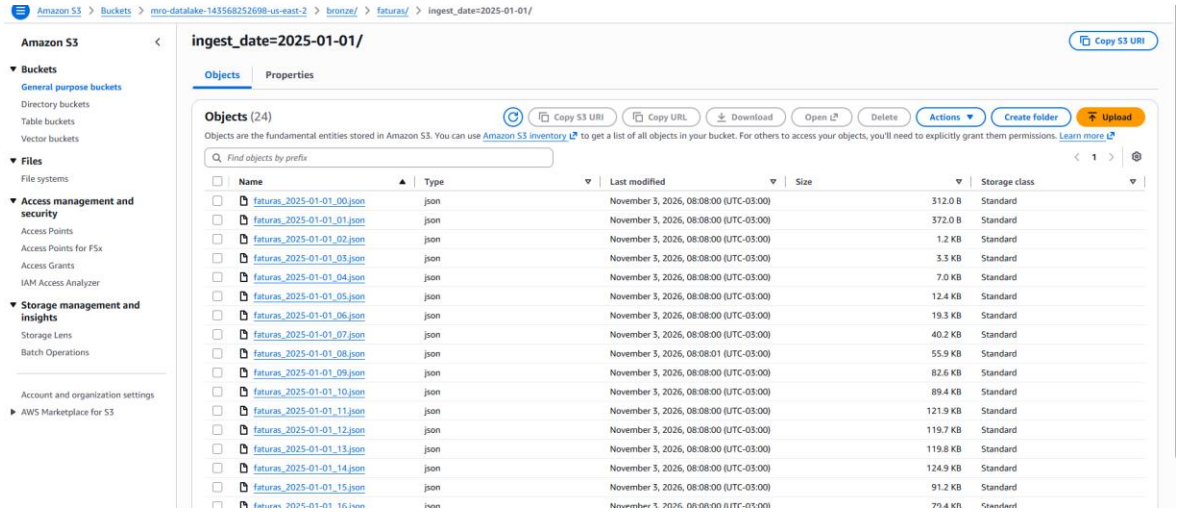


Figura 9 – Uma partição da Bronze com 24 arquivos JSON pequenos (imagem 11)

Ingestão e Transformação

Configuração no AWS Glue Data Catalog com Crawlers

O crawler `crawler_mro_bronze` descobriu o esquema dos arquivos e criou 3 tabelas e 181 partições em 52 segundos. Depois, um script completou os metadados técnicos: descrição de cada tabela, comentário de cada coluna e parâmetros de governança (camada, sensibilidade, presença de dado pessoal, fonte e granularidade).

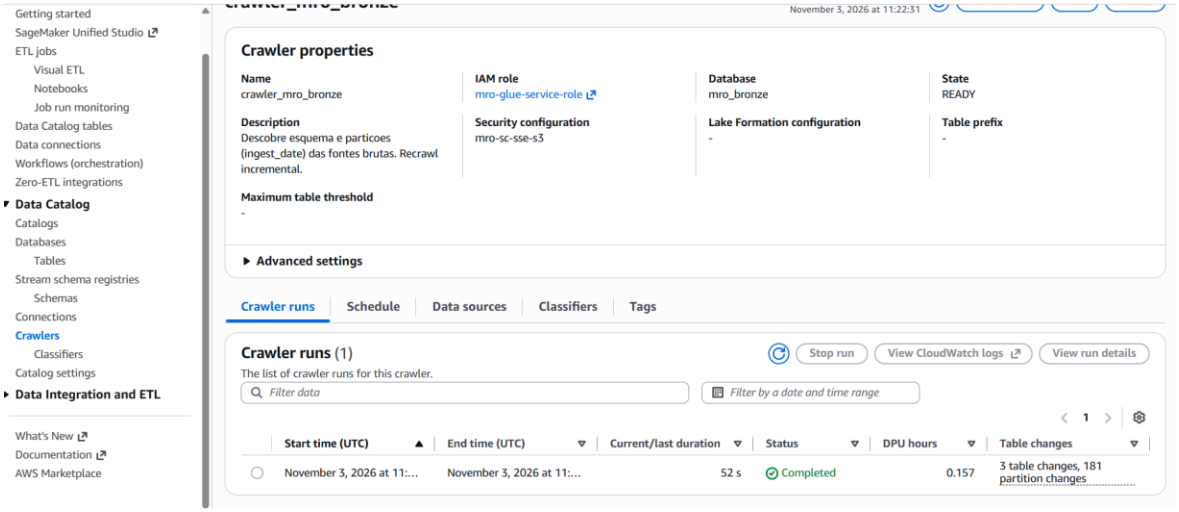


Figura 10 – Execução do crawler da Bronze: 52 s, 3 tabelas e 181 partições criadas (imagem 03)

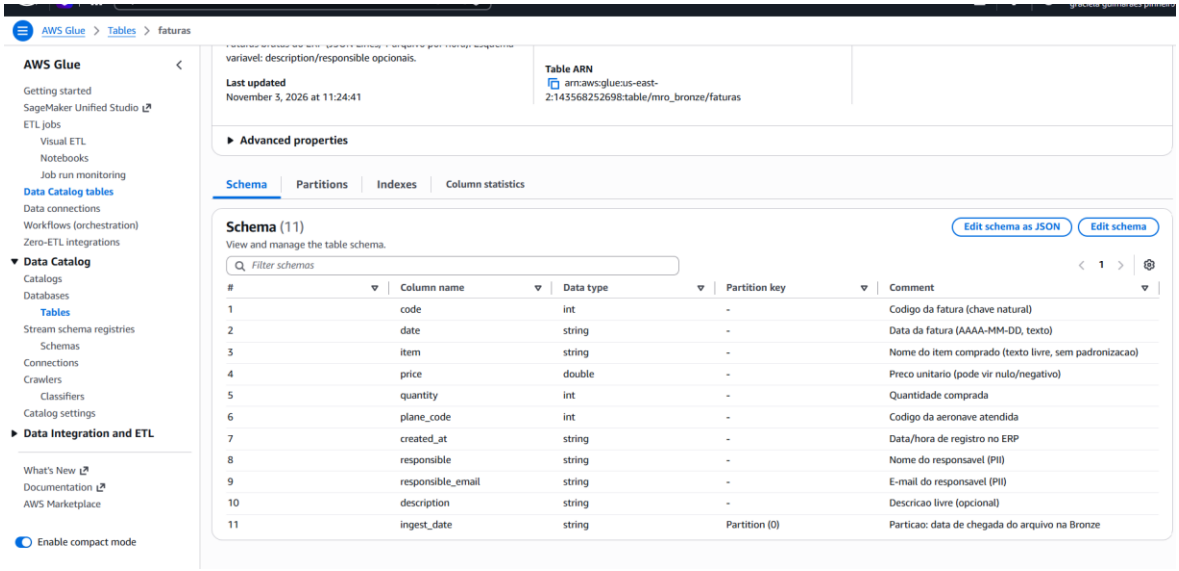


Figura 11 – Schema de mro_bronze.faturas com tipos, partição e comentários das colunas (imagem 04)

Fonte JDBC: cadastros em um banco RDS PostgreSQL

Para ter uma fonte relacional, como acontece com um sistema de cadastro real, criei com CloudFormation a stack `glue-pd-jdbc`: um banco **RDS PostgreSQL** (db.t4g.micro, sem acesso público, criptografado), a conexão JDBC **mro-rds-postgres** no Glue e endpoints de VPC para o S3, o Glue, o CloudWatch Logs e o Secrets Manager. A senha do banco não aparece em nenhum lugar: ela fica no **Secrets Manager** e a conexão só guarda a referência ao segredo.

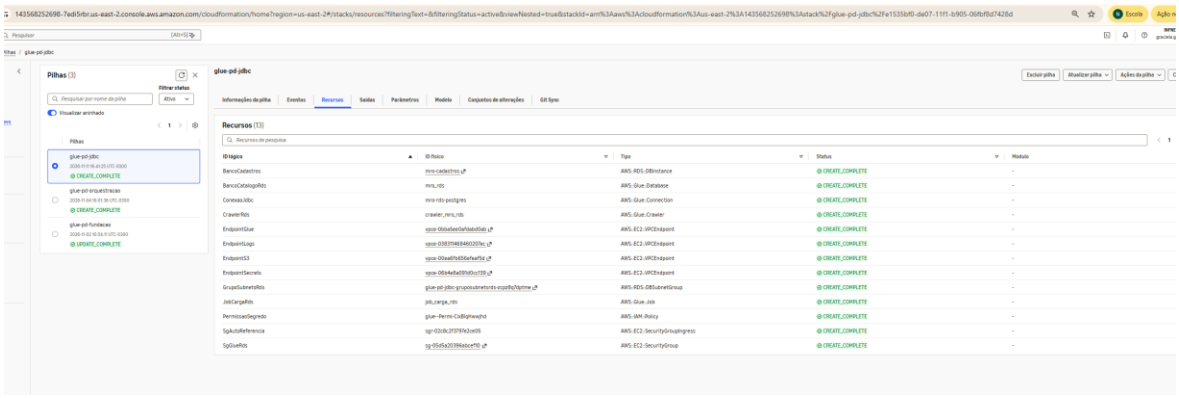


Figura 12 – Stack glue-pd-jdbc com os 13 recursos criados: banco, conexão, crawler, job, endpoints e security group (imagem 64)

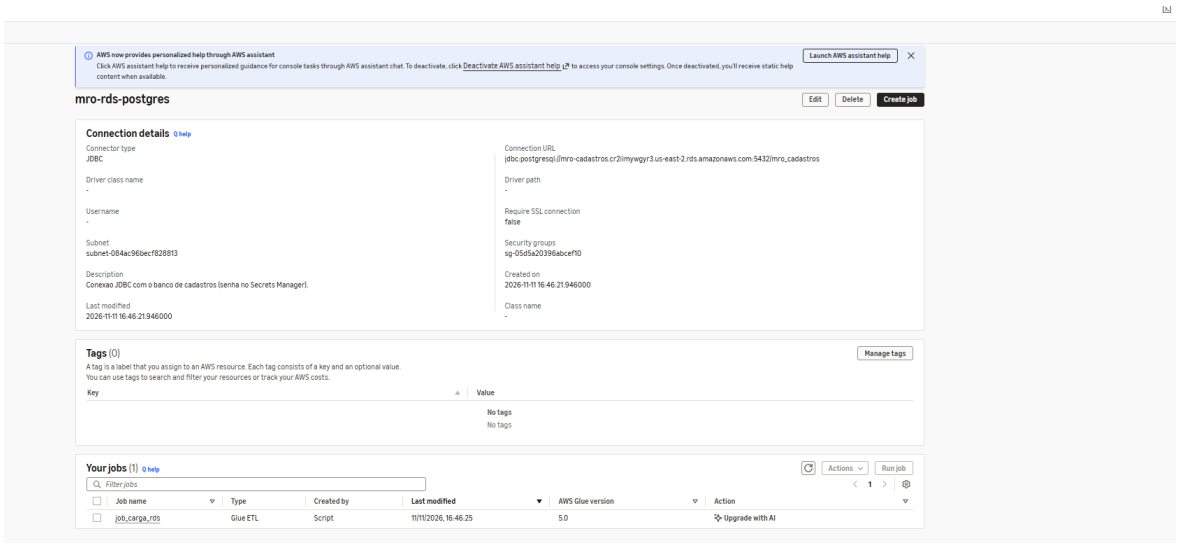


Figura 13 – Conexão JDBC mro-rds-postgres: URL do PostgreSQL, subnet e security group, sem usuário e senha gravados (imagem 67)

O job `job_carga_rds` carregou os produtos e os aviões no banco pela conexão (2 min 1 s). Em seguida, o crawler JDBC `crawler_mro_rds` registrou as duas tabelas no novo banco `mro_rds` do Data Catalog, com a classificação **postgresql**. Por fim, troquei a origem do job visual de dimensões, que antes lia a Bronze, para essas tabelas JDBC, e ele rodou com sucesso em 2 min 15 s.

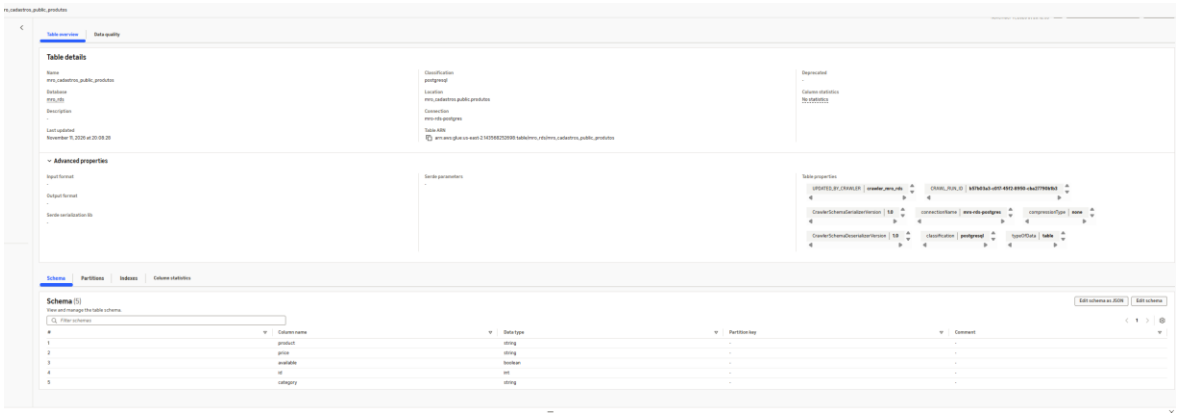


Figura 14 – Tabela JDBC mro_cadastros_public_produtos no Catalog: classificação postgresql, local banco.schema.tabela e conexão (imagem 70)

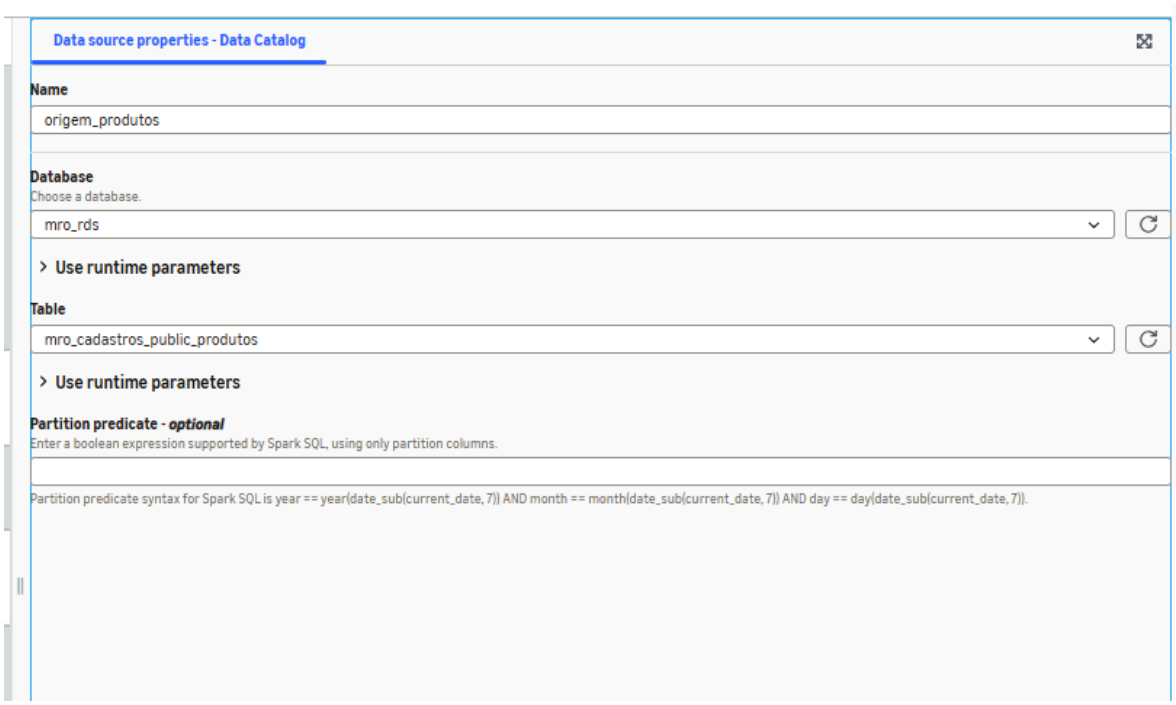


Figura 15 – Job visual de dimensões com a origem trocada para o banco mro_rds (imagem 73)

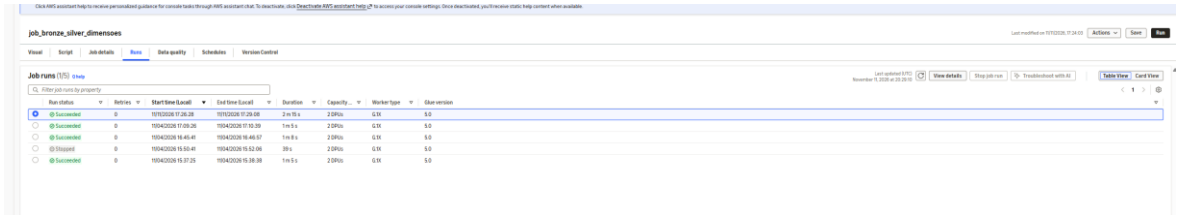


Figura 16 – Execução do job de dimensões lendo do RDS pela conexão JDBC (imagem 74)

Configuração no AWS Glue de algumas transformações

O job visual `job_bronze_silver_faturas` tem 14 nós. Ele renomeia e tipa as colunas (Change Schema), cria a chave de junção e as colunas de ano e mês (Derived Column), separa os registros inválidos numa quarentena (SQL Query), filtra os válidos (Filter), remove duplicados pelo código da fatura (Drop Duplicates), aplica hash no e-mail, redistribui os dados (Autobalance) e avalia a qualidade (Evaluate Data Quality) antes de gravar.

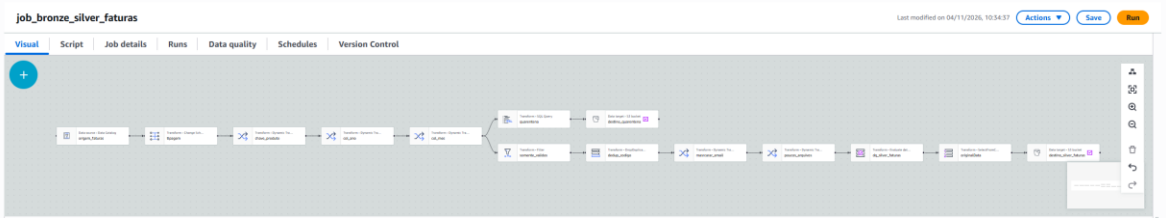


Figura 17 – Grafo do job visual job_bronze_silver_faturas no Glue Studio (imagem 23)

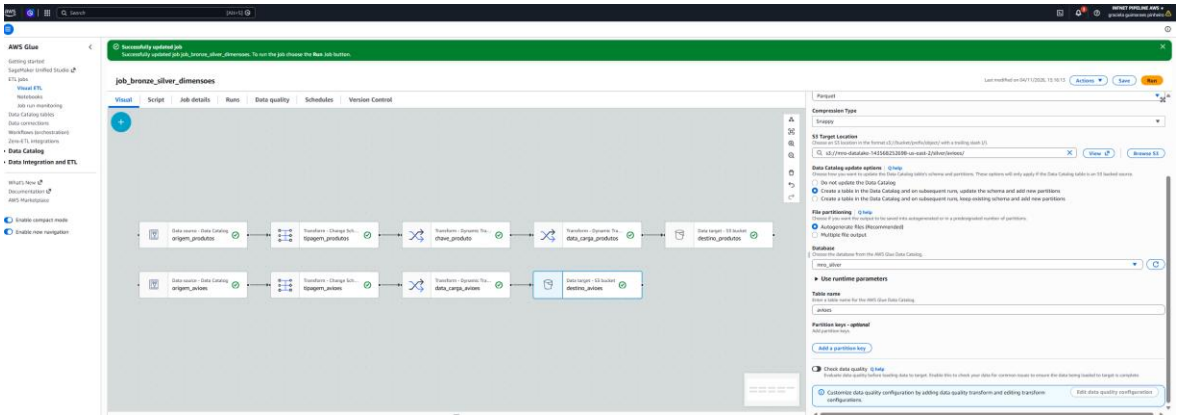


Figura 18 – Grafo do job visual job_bronze_silver_dimensoes, com os ramos de produtos e aviões (imagem 27)

Os jobs também são **parametrizados**: os nomes do bucket, dos bancos e dos prefixos não ficam fixos no código, e sim nos parâmetros do job (`--BUCKET`, `--DB_SILVER`, `--PREFIXO_GOLD`, `--CONEXAO`). Assim, o mesmo script pode rodar em outro ambiente só trocando os parâmetros no CloudFormation.

Configuração no AWS Glue convertendo para Parquet

Os nós de destino gravam em Parquet com Snappy e criam ou atualizam a tabela no Data Catalog a cada execução.

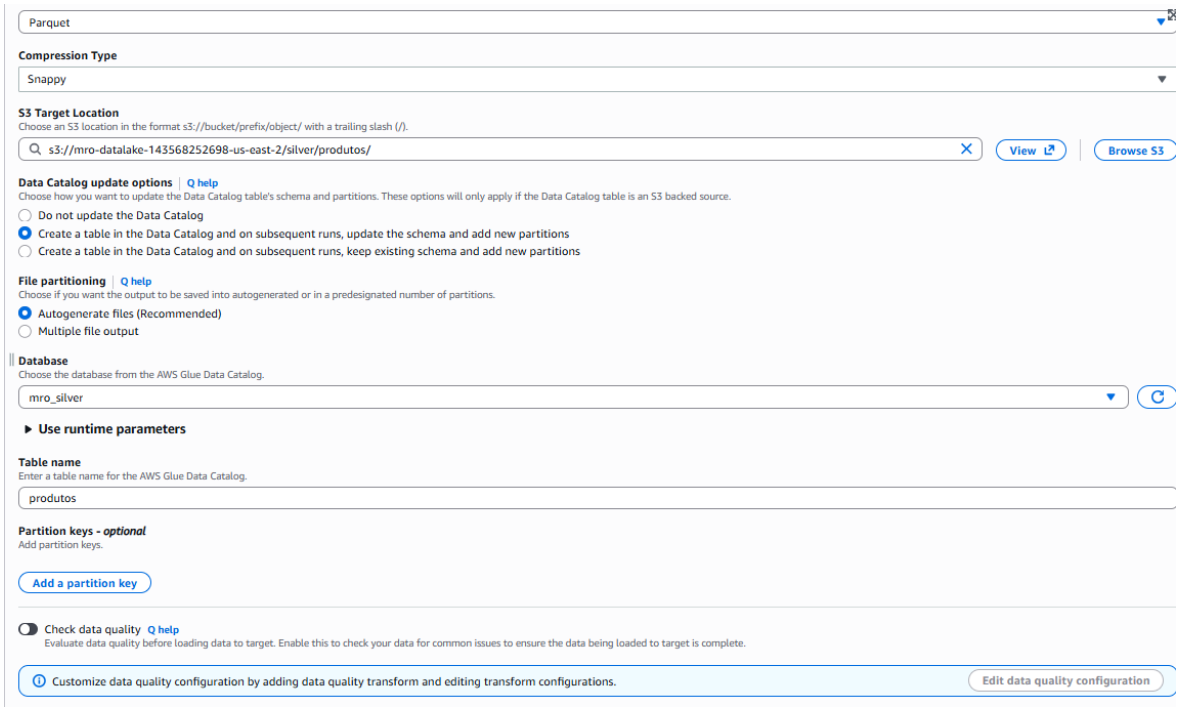


Figura 19 – Destino S3 em Parquet/Snappy, criando a tabela mro_silver.produtos no Catalog (imagem 26)

Eficiência e Performance

Configuração no AWS Glue do Job Bookmarks

O Job Bookmark do job de faturas está habilitado. Na primeira carga, o job levou 7 min 55 s; na segunda, com só o lote 2, levou 3 min 42 s, porque leu apenas os arquivos novos. Aprendi

também um detalhe: o bookmark processa no máximo 1.000 arquivos recém-modificados por execução (opção `maxFilesInBand`); os 467 restantes do lote 2 entraram na execução seguinte, sem perda de dados.

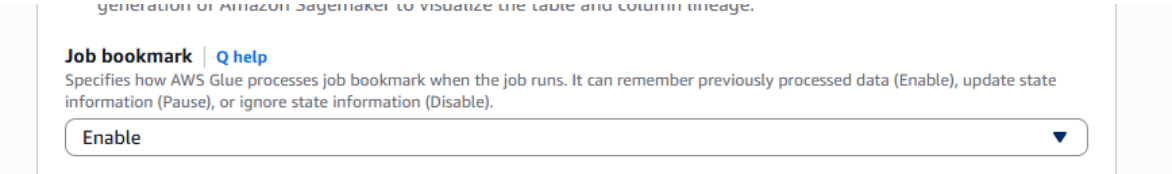


Figura 20 – Job Bookmark habilitado (imagem 13)

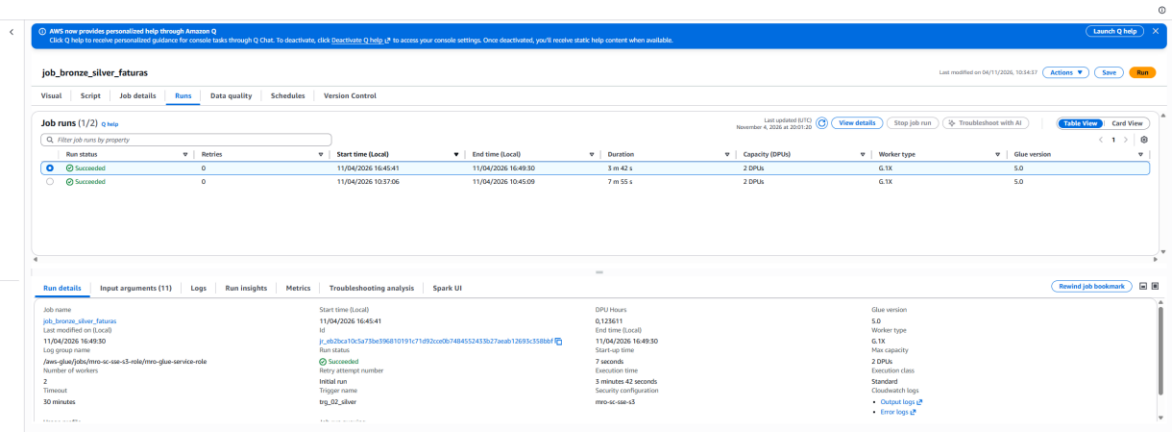


Figura 21 – Execuções do job de faturas: lote 1 (7 min 55 s) e lote 2 só com arquivos novos (3 min 42 s) (imagem 37)

Captura de mudanças (CDC) da fonte JDBC

O bookmark resolve os arquivos novos, mas um cadastro em banco muda de outro jeito: entra produto, muda preço, sai produto. Para isso fiz um CDC por comparação: o job `job_cdc_produtos` lê o cadastro atual no PostgreSQL, compara com a última versão de cada produto na Silver e classifica cada diferença como **I** (inclusão), **U** (alteração) ou **D** (exclusão). As mudanças vão para a tabela de log `mro_silver.cdc_produtos` e uma nova versão dos produtos alterados é gravada na Silver, que a Gold já usa automaticamente (sempre a versão mais recente).

Para testar, o job `job_simula_alteracoes_rds` fez o papel do sistema da empresa: incluiu 3 produtos, reajustou o preço de 2 e excluiu 1. O CDC capturou exatamente essas 6 mudanças. O produto excluído não é apagado da Silver, porque faturas antigas apontam para ele; ele recebe `disponivel = false` (exclusão lógica). Em um ambiente maior, eu usaria o AWS DMS, que lê o log do próprio banco, mas para um cadastro pequeno a comparação é simples e barata.

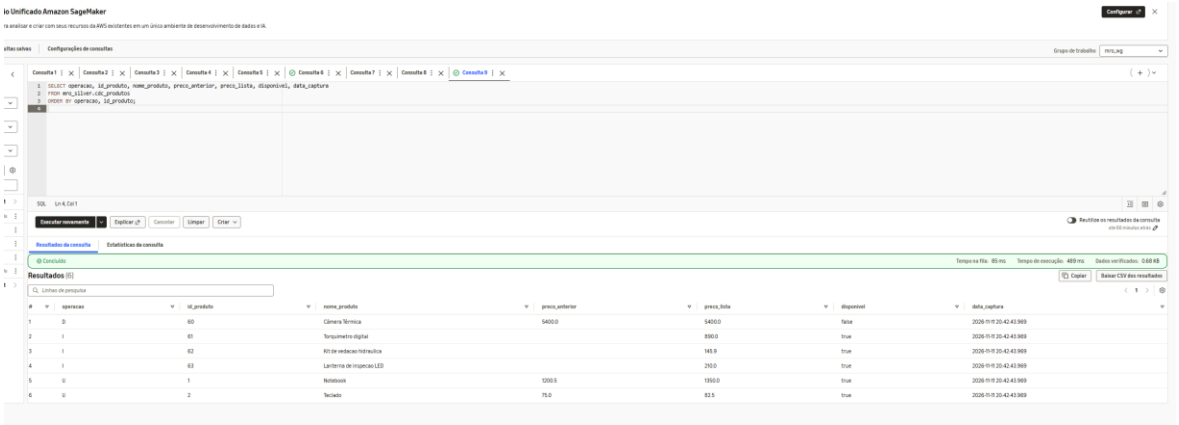


Figura 22 – Log do CDC no Athena: 1 exclusão (D), 3 inclusões (I) e 2 alterações de preço (U) (imagem 77)

Configuração do agendamento dos Jobs

O agendamento e o encadeamento ficam no Glue Workflow `wf_mro_pipeline`. O gatilho inicial dispara o crawler da Bronze; em seguida rodam os dois jobs da Silver em paralelo; o job Gold só começa quando os dois terminam com sucesso (lógica AND); por fim, o crawler da Gold atualiza o Catalog. O gatilho inicial está como sob demanda, para controlar o custo durante o trabalho; para rodar todo dia, ele seria trocado por um gatilho agendado, como `cron(0 6 * * ? *)`.

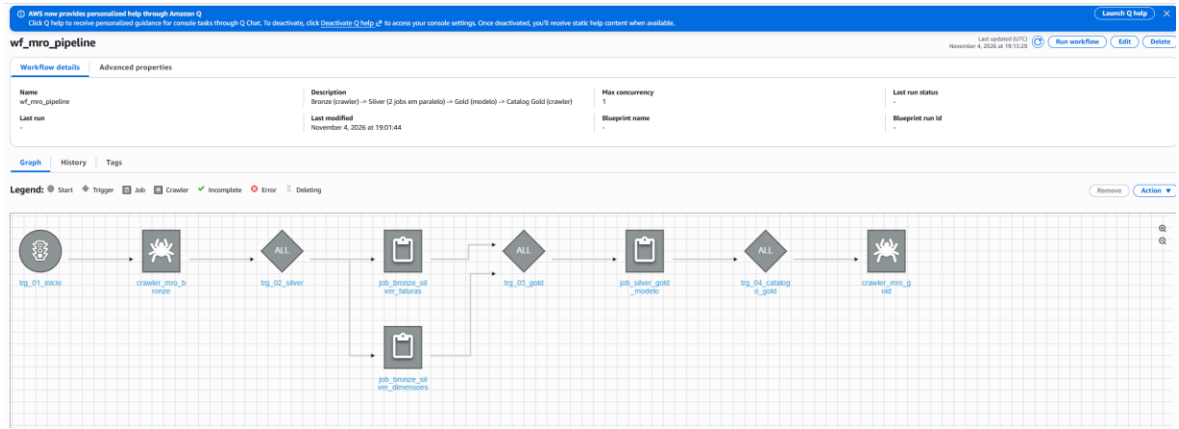


Figura 23 – Grafo do workflow wf_mro_pipeline com os gatilhos encadeados (imagem 33)

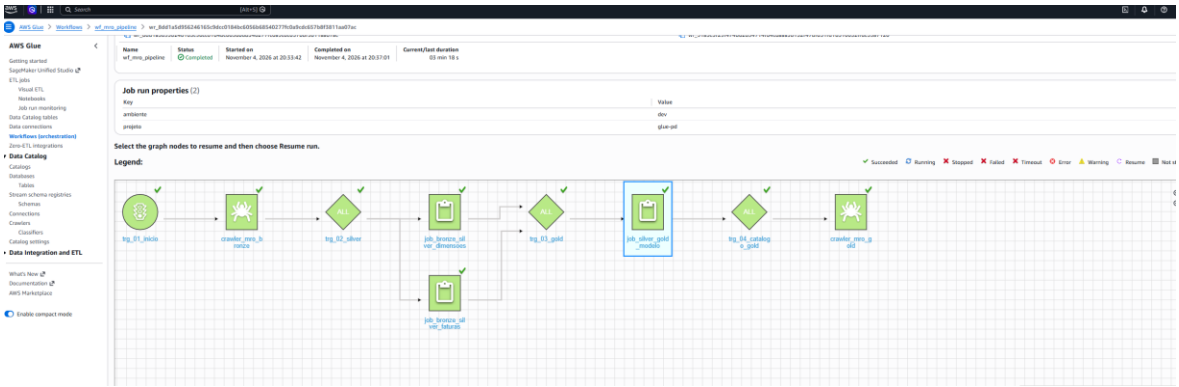


Figura 24 – Execução do workflow concluída, com todos os nós em verde (imagem 42)

Governança, segurança e qualidade dos dados

Configuração da proteção de informações confidenciais

O e-mail do responsável é um dado pessoal. Primeiro usei o nó Detect Sensitive Data, mas ele converteu todas as colunas para texto e quebrou as regras de qualidade. Troquei por uma coluna derivada com `sha2(email_responsavel, 256)`, que substitui o e-mail por um hash irreversível. No Athena, o e-mail aparece só como hash.

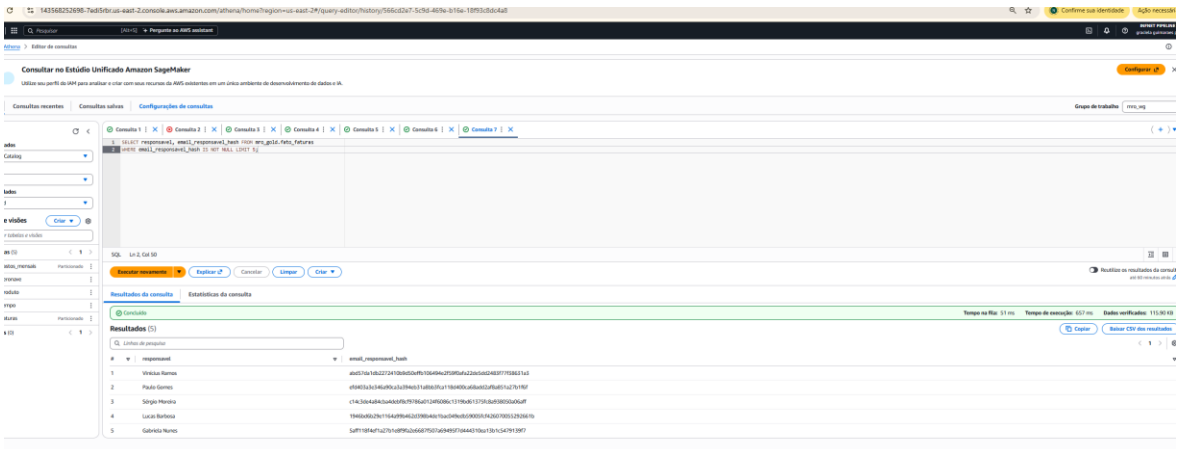


Figura 25 – Consulta no Athena: o e-mail do responsável aparece apenas como hash SHA-256 (imagem 48)

No AWS Glue e AWS Glue DataBrew

A qualidade foi validada com o Glue Data Quality, dentro do próprio job, com 10 regras (completude, unicidade, faixas de valores). O resultado foi 9 de 10: a regra que falha aponta as 2.383 faturas com aeronave inexistente, que foram mantidas e ligadas a um membro "não identificado" na Gold.

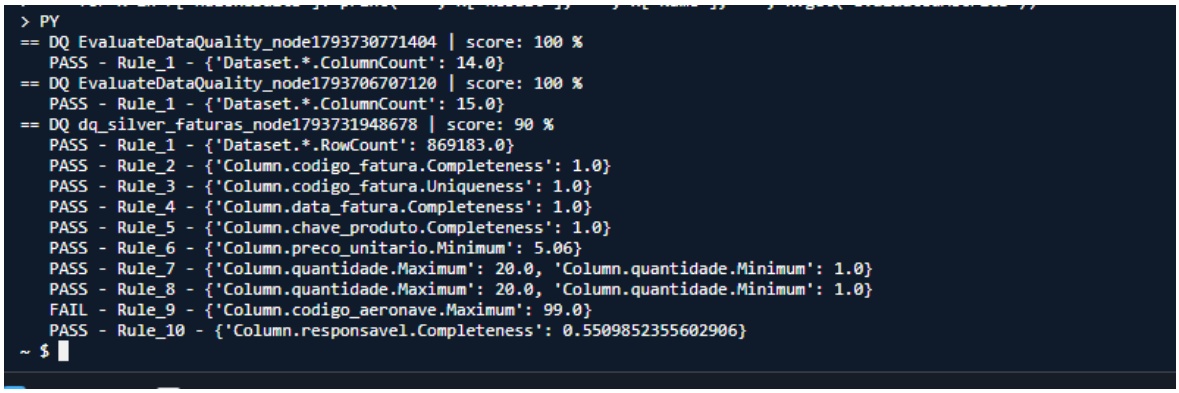


Figura 26 – Resultado do Data Quality: 9 de 10 regras aprovadas e 869.183 registros avaliados (imagem 22)

No **Glue DataBrew**, criei o conjunto de dados `bronze-faturas` direto do Data Catalog e rodei um trabalho de perfil com as estatísticas de **PII** ligadas, usando uma role própria (`mro-databrew-role`) que só lê a Bronze e a Silver e só grava na pasta `databrew/`. Numa amostra de 20.000 linhas, o perfil mostrou 18% de células vazias e 8 linhas duplicadas, e identificou dados pessoais em 4 das 10 colunas: `responsible_email` como **e-mail**, `responsible` como **nome**, `date` como data e `item` como nome. Este último é um falso positivo (são nomes de ferramentas), o que mostra que a detecção automática ajuda, mas precisa de conferência. O resultado confirmou a decisão de aplicar hash no e-mail.

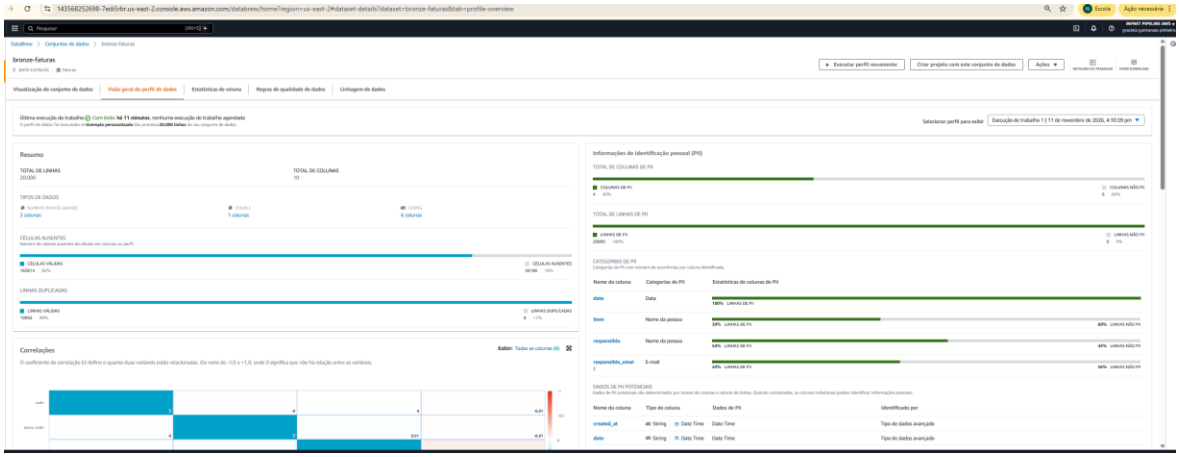


Figura 27 – Perfil do DataBrew com a identificação automática de PII em 4 colunas (imagem 58)

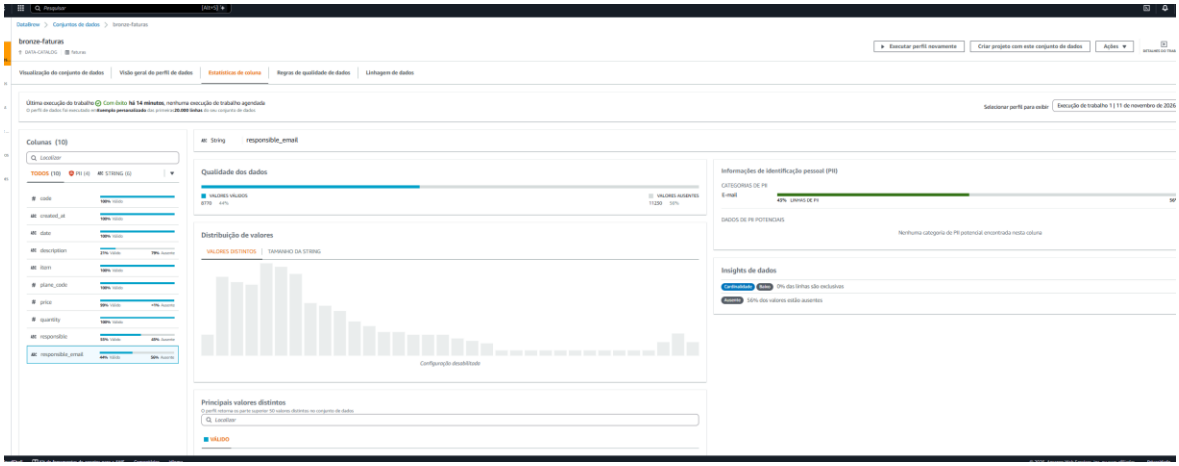


Figura 28 – Coluna responsible_email identificada como e-mail (imagem 59)

Depois montei a receita `limpeza-faturas`, com 3 etapas: remover espaços em branco de `item`, passar `item` para minúsculas e excluir as linhas sem preço. O trabalho `limpeza-bronze-faturas` aplicou a receita em 7 min 54 s e gravou 181 arquivos Parquet (22,1 MiB) em `databrew/limpeza/`. A aba de linhagem mostra o caminho completo: conjunto de dados, receita, trabalho e saída no S3.

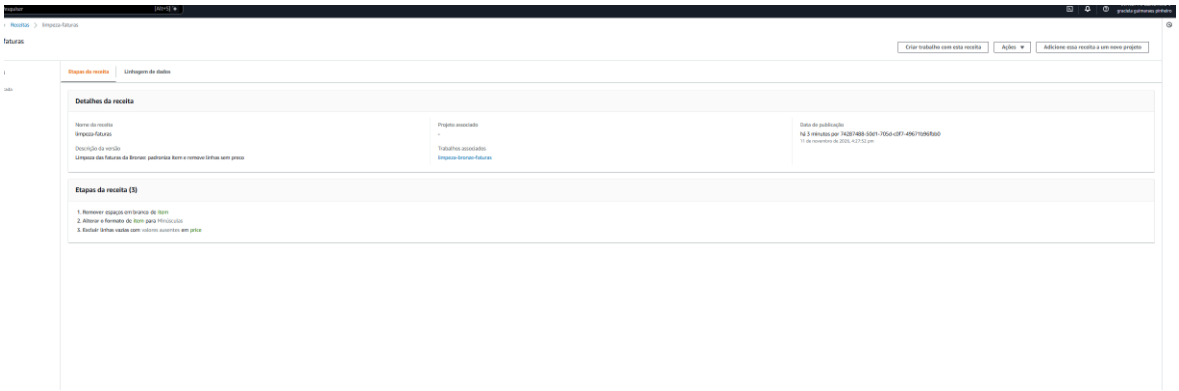


Figura 29 – Receita limpeza-faturas com as 3 etapas de limpeza (imagem 61)

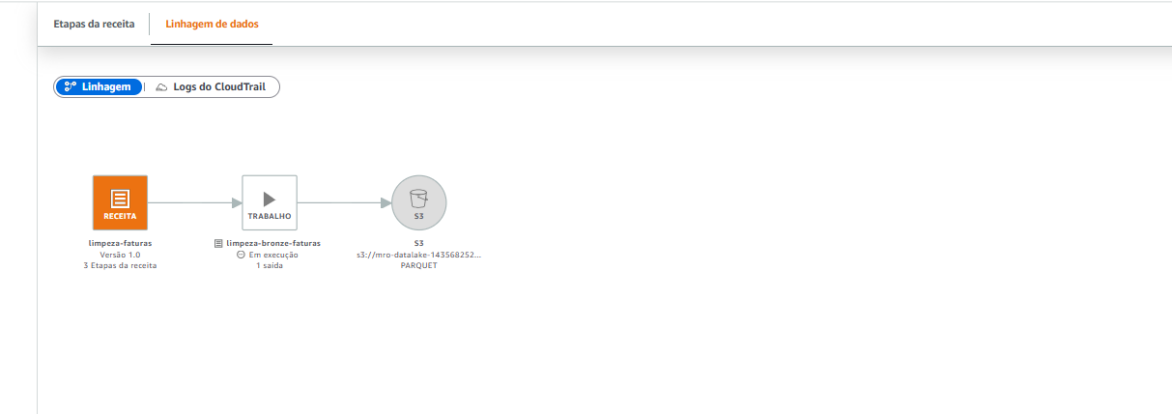


Figura 30 – Linhagem no DataBrew: conjunto de dados, receita, trabalho e saída Parquet no S3 (imagem 62)

Conclusão

O pipeline rodou de ponta a ponta: 1.198.606 faturas chegaram à Silver e à Gold, os registros inválidos ficaram na quarentena (4.799 com preço nulo e 2.340 com preço negativo) e as consultas no Athena bateram com os valores esperados. O custo real ficou em poucos dólares, e a maior parte veio das sessões de pré-visualização do editor visual, que eu não sabia que cobravam por hora. O banco RDS e os endpoints da parte JDBC custam cerca de US$ 0,05 por hora, por isso a stack foi apagada logo depois dos testes.

Os maiores aprendizados vieram dos erros: uma política da organização que bloqueava grupos IAM, um nó que mudava os tipos das colunas, uma tabela classificada errado pelo crawler, o limite de arquivos do bookmark e uma permissão que faltava para o Spark sobrescrever a Gold. Cada um foi diagnosticado e corrigido. Na parte final, o DataBrew, a fonte JDBC e o CDC me mostraram que o mesmo pipeline pode receber dados de arquivos e de bancos, e que a governança (senha no Secrets Manager, detecção de PII, log de mudanças) precisa acompanhar cada nova fonte.

Referências

- AWS. AWS Glue Developer Guide. Disponível em: https://docs.aws.amazon.com/glue/latest/dg/
- AWS. Tracking processed data using job bookmarks. https://docs.aws.amazon.com/glue/latest/dg/monitor-continuations.html
- AWS. AWS Glue Data Quality. https://docs.aws.amazon.com/glue/latest/dg/glue-data-quality.html
- AWS. AWS Glue pricing. https://aws.amazon.com/glue/pricing/
- AWS. Amazon Athena pricing. https://aws.amazon.com/athena/pricing/
- AWS. AWS Glue DataBrew Developer Guide. https://docs.aws.amazon.com/databrew/latest/dg/
- AWS. AWS Glue JDBC connections. https://docs.aws.amazon.com/glue/latest/dg/connection-properties.html